

mvp_GAP_AUDIT

PHASE MVP — Code-Grounded Gap Audit

Honest assessment: ~60% complete | Auditor: engineering review | 2026-06-01

One-line summary: Cluster-formation primitives (SWIM/WireGuard/discovery/leader/etcd/NATS), the resource readers, the Omega scheduler core, and the security crypto layer are genuinely implemented with real-behavior tests; but the **Build Service is simulated, Kafka pub/sub is in-memory only, session I/O has no remote WebSocket forwarding, the API gateway has no auth**, node state is ephemeral in-memory, and there is no HelixQA Challenge runner that exercises live services.

The roadmap/MVP_PROGRESS docs overstate stubs (events/etcd are now real) **and** overstate completion (build, pubsub, session-forwarding, gateway-auth are weaker than implied). This audit corrects both.

3-Axis Package Status (Refreshed 2026-06-04, HXC-939)

Why this section exists: “exists” ≠ “used”. A package can be **implemented** and **tested** yet never reached from any binary (**orphaned**). wired is **measured** via `go list -deps ./cmd/... | grep -Fx <module-path>/<pkg>` (module = github.com/HelixDevelopment/helix_cluster), not assumed. **“Completed (registry) ≠ wired”** — Completed only means source+tests exist; it does NOT prove a shipped binary reaches it.

STALENESS CORRECTION: the original table called pkg/gpuattest an “empty dir / dangling placeholder”. As of 2026-06-02 it is a **real, populated, tested local package** (attest/povw/seal/spotcheck/multigpu) — implemented + tested, but **orphaned** (no binary imports it). Row below is authoritative.

Package	implemented	wired (reachable from cmd/)	tested
pkg/events (NATS)	yes	NO (orphaned)	yes
pkg/etcd	yes	yes	yes

Package	implemented	wired (reachable from cmd/)	tested
pkg/ discovery (etcd backend)	yes	yes	yes
pkg/pubsub (in-memory only)	yes	NO (orphaned)	yes
pkg/ resources (cgroup v2 + /proc)	yes	yes	yes
pkg/ scheduler (Omega core)	yes	yes	yes
internal/ scheduler	yes	yes	yes
pkg/session (+ backends)	yes	yes	yes
internal/ node / cmd/ helix-node	yes	yes	yes
pkg/ wireguard	yes	yes	yes
internal/ gateway / cmd/helix- gateway	yes	yes	yes
pkg/swim	yes	yes	yes
pkg/leader	yes	NO (orphaned)	yes
pkg/lock	yes	NO (orphaned)	yes
pkg/ hxcregistry (Postgres, nested module)	yes	NO (orphaned)	yes
internal/ gpu	yes	yes	yes
internal/ build / cmd/ helix-build (simulated build)	yes	yes	yes
internal/ security (RBAC scopes)	yes	yes	yes

Package	implemented	wired (reachable from cmd/)	tested
cmd/htmux	yes	yes	yes
internal/ session / cmd/helix- session	yes	yes	yes
pkg/metrics	yes	yes	yes
pkg/tracing	yes	NO (orphaned)	yes
cmd/helix- test (HelixQA-ish runner)	yes	yes	yes
pkg/ gputest (local; was “empty”)	yes	NO (orphaned)	yes
pkg/jwt	yes	yes	yes
pkg/ middleware	yes	NO (orphaned)	yes
pkg/ websocket	yes	yes	yes
security/ pkg/e2ee / attestation (separate module)	yes	NO (not in helix cmd graph)	yes

Orphan callouts (implemented+tested but reachable from NO cmd/ binary): pkg/events, pkg/pubsub, pkg/leader, pkg/lock, pkg/hxcregistry, pkg/tracing, pkg/gputest, pkg/middleware. Several of these are precisely the MVP gaps the prose already names (e.g. pkg/middleware+pkg/jwt exist but the gateway doesn’t enforce auth → pkg/middleware orphaned; node registry not etcd-backed → relevant backends sit unwired). pkg/websocket IS now wired (the old “not wired to session attach” note may be partially stale at the package level — verify the *feature* path separately).

Methodology

DONE = real implementation **and** real-behavior test (unit + integration vs real service where the feature claims real-world operation). PARTIAL = implemented but simulated/mock-only/missing a load-bearing path. MISSING = absent or empty. Integration tests are gated //go:build integration and boot **real** containers via containers/pkg/brokertest (StartNATS/StartEtcd/StartPostgres, real testcontainers).

Deliverable Status Table

Deliverable	Status	Evidence (file:line)
P0-1 NATS/JetStream client (pkg/events)	DONE	pkg/events/nats_backend.go:8; pkg/events/nats_integration_test.go:1 (//go:build integrati broker)
P0-2 etcd client (pkg/etcd)	DONE	pkg/etcd/package.go:9 (clientv3); pkg/etcd/etcd_integration_test.go:1 (real etcd)
etcd discovery backend (pkg/discovery)	DONE	pkg/discovery/etcd_backend.go:12,52; etcd_backend_test.go
P0-3 Kafka producer/consumer (pkg/pubsub)	PARTIAL	pkg/pubsub/package.go:6–36
P0-4 cgroups v2 + /proc reader (pkg/resources)	DONE	pkg/resources/cgroup_v2.go:96,148,187; proc_linux.go:22,39,92; cgroup_v2_test.go + testc
P0-5 Omega scheduler core (pkg/scheduler, internal/scheduler)	DONE	pkg/scheduler/scheduler.go:101,157 (ScheduleOpt w/ version); cost_gpu.go; gang_preempt.go; internal/ scheduler/server.go
P0-6 tmux control-mode backend (pkg/session)	PARTIAL	pkg/session/backends/tmux.go:20,34,45
P0-7 Node gRPC handlers (internal/node, cmd/helix-node)	PARTIAL	internal/node/server.go:20–31,36 (in-memory maps) helix-node/main.go:110,115 (real grpc.NewServer)
P0-8 WireGuard mesh + NAT traversal (pkg/wireguard)	PARTIAL	pkg/wireguard/mesh.go, configgen.go, keyrotation. nat_traversal.go:17–19

Deliverable	Status	Evidence (file:line)
P0-9 API gateway wiring (internal/gateway,cmd/helix-gateway)	PARTIAL	internal/gateway/gateway.go:27-30,83,91; cmd/helix-gateway/main.go:117
SWIM gossip (pkg/swim)	DONE	pkg/swim/transport.go:14,21 (real net.ListenUDP); failure_detector.go,suspicion.go,prober.go
Leader election + fencing (pkg/leader)	DONE	pkg/leader/fencing.go:64,117,155
Distributed lock (pkg/lock)	DONE	pkg/lock/lock_integration_test.go (//go:build integration, real etcd)
Internal artifact registry (pkg/hxcregistry)	DONE	pkg/hxcregistry/registry.go,schema.sql,hxcregistry_integration_test.go (real Postgres)
GPU resource probe (internal/gpu)	PARTIAL	internal/gpu/manager.go:36,52,57,118
Build Service (internal/build,cmd/helix-build)	PARTIAL/ SIMULATED	internal/build/orchestrator.go:366-390 (runBuild time.After(100ms) then fake ImageTag); worker.go (no
e2ee transport (security/pkg/e2ee)	DONE	security/pkg/e2ee/package.go:21 (AES-256-GCM); transport.go:40,62; package_test.go

Deliverable	Status	Evidence (file:line)
Attestation (security/pkg/attestation)	PARTIAL (honest)	security/pkg/attestation/attestation.go:14–22,8
RBAC scopes / identity (internal/ security)	DONE	internal/security/identity_bindings.go; authorize_rbac_test.go; scopes_test.go; policy_enforcer.go
htmux CLI (cmd/ htmux)	PARTIAL	cmd/htmux/main.go:27; terminal.go:84,134 (comment “session service does not expose a streaming endpoint”)
Session I/O WebSocket forwarding (cmd/ helix-session)	MISSING	internal/session/* + pkg/session/manager.go:28,2 Stream/WebSocket)
HelixQA Challenge runner (cmd/helix- test)	PARTIAL	cmd/helix-test/main.go:5–7,86,117 (dst/chaos/device
Prometheus metrics / tracing (pkg/ metrics,pkg/ tracing)	PARTIAL	pkg/metrics/*, pkg/tracing/* (real helpers)
GPU attestation glue (pkg/gpuattest)	MISSING (empty)	pkg/gpuattest/ (no files)

TOP IMPLEMENTABLE GAPS (Go, no new infra)

1. **Session I/O WebSocket forwarding** — `internal/session + cmd/helix-session (+ reuse pkg/websocket)`. Add a `StreamSession/attach` handler that bridges a tmux pane's PTY to a WebSocket: spawn `tmux -CC` (or `pipe-pane`) for the named session, copy bytes bidirectionally between the PTY fd and the WS conn, and propagate `resize`. Expose it on the session service and have `cmd/htmux/terminal.go:134` dial it. *Acceptance test (mutation-pairable)*: loopback WS test — write `"echo hi\n"` to the client side, assert `"hi"` echoes back within 1s; **mutation**: drop the server→client copy goroutine ⇒ test must hang/fail.
2. **API gateway auth middleware** — `internal/gateway`. Wrap the reverse-proxy handler with `pkg/middleware + pkg/jwt`: reject requests lacking a valid Bearer token (401), inject verified claims, and enforce `internal/security` scopes per route prefix. *Acceptance*: request to `/api/v1/scheduler/` with no/invalid/expired token ⇒ 401; valid token + correct scope ⇒ proxied 200; **mutation**: make the verifier always return nil error ⇒ the unauthorized cases must start failing.
3. **Kafka producer/consumer** — `pkg/pubsub`. Replace/extend the in-memory Broker with a real client (`segmentio/kafka-go`) implementing `Publish(topic, key, value) / Subscribe(topic, group)` with at-least-once delivery and offset commit; keep the in-memory impl behind the same interface for unit tests. *Acceptance (integration, real Kafka via brokertest pattern)*: publish N messages, consume across 2 group members, assert every key delivered exactly to one consumer; **mutation**: skip the offset commit ⇒ redelivery assertion fails.
4. **Etcd-backed node registry** — `internal/node`. Persist `RegisterNode/UpdateNodeStatus` into `pkg/etcd` (lease-bound keys, TTL = heartbeat window) instead of the in-memory map, and read `ListNodes` from etcd so two `helix-node` processes see each other. *Acceptance (integration, real etcd)*: register on instance A, `ListNodes` on instance B returns it; stop A's keepalive ⇒ entry expires within TTL; **mutation**: never attach the lease ⇒ the expiry assertion fails.
5. **HelixQA live-service Challenge runner** — `cmd/helix-test` (challenge subcommand). Add a runner that boots a service (or compose stack), drives the real endpoint, and writes captured sink-side evidence (response/log/metric) to an artifact dir, exiting non-zero on mismatch. Wire 3–5 challenges/ dirs to it. *Acceptance*: a passing challenge produces an evidence file with the expected sink line; **mutation**: point the challenge at a wrong endpoint ⇒ runner exits non-zero and writes a failure record.
6. **tmux control-mode (-CC) backend** — `pkg/session/backends/tmux.go`. Upgrade `Attach` to use `tmux -CC` control mode, parsing `%output/%begin/%end` notifications into structured events so the session service can multiplex panes. *Acceptance*: create a session, send a command, assert a parsed `%output` event contains the command's stdout; **mutation**: ignore `%output` frames ⇒ assertion fails.

7. **Uniform metrics/health wiring across services** — cmd/helix-*. Mount pkg/metrics /metrics and pkg/health on every binary's HTTP/gRPC server.
Acceptance: table test hitting each command's /metrics returns a helix_* counter that increments after one request; **mutation:** drop the counter .Inc() in a handler ⇒ the increment assertion fails.

DEFERRED (external infra / hardware / non-Go)

- **Bazel RBE / distcc real build execution** (internal/build) — DEFERRED: needs a real Bazel/AOSP toolchain and remote-execution cluster; current simulated runBuild is acceptable only as a placeholder, not MVP-complete.
- **Hardware-rooted attestation (TDX/SGX/NVIDIA)** (security/pkg/attestation, pkg/gpuattest) — DEFERRED: requires confidential-compute hardware; software ed25519 stand-in is honestly documented.
- **Real GPU probe validation** (internal/gpu) — DEFERRED for sink-side proof: needs an actual NVIDIA GPU; /proc parser is real but unverifiable in CI.
- **WireGuard NAT traversal / STUN hole-punch** (pkg/wireguard/nat_traversal.go) — DEFERRED: needs a STUN/TURN relay + multi-host NAT topology to validate; placeholder HTTP discovery insufficient.
- **CRIU live migration, Zellij/screen backends, Zig primitives, C/C++ GPU kernels, PS4/PS5 console nodes** — DEFERRED per roadmap §6 P2 (non-Go and/or hardware-bound).
- **Vault secret rotation + Grafana scrape exit-gates** — DEFERRED for sink-side evidence: needs the running observability/secrets stack (compose) to capture operator-visible proof.